Strings in C

Strings in C

A **string in C** is a one-dimensional array of char type, with the last character in the array being a "null character" represented by '\0'. Thus, a string in C can be defined as a null-terminated sequence of char type values.

Creating a String in C

Let us create a string "Hello". It comprises five char values. In C, the literal representation of a char type uses single quote symbols – such as 'H'. These five alphabets put inside single quotes, followed by a null character represented by '\0' are assigned to an array of char types. The size of the array is five characters plus the null character – six.

Example

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

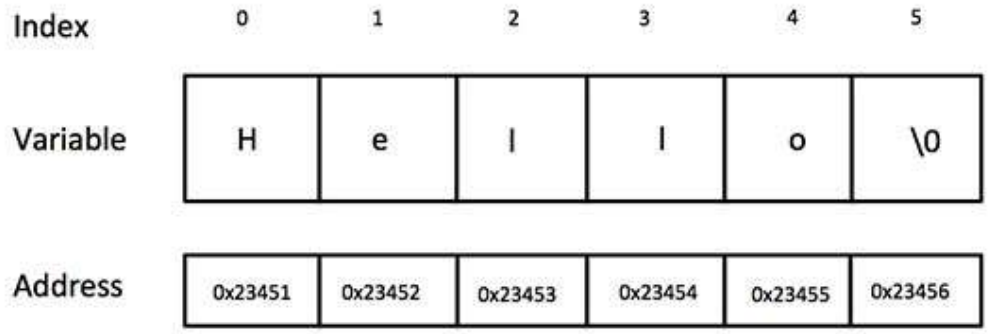
Initializing String Without Specifying Size

C lets you initialize an array without declaring the size, in which case the compiler automatically determines the array size.

Example

```
char greeting[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

The array created in the memory can be schematically shown as follows –



If the string is not terminated by "\0", it results in unpredictable behavior.

Note: The length of the string doesn't include the null character. The library function strlen() returns the length of this string as 5.

Loop Through a String

You can loop through a string (character array) to access and manipulate each character of the string using the for loop or any other loop statements.

Example

In the following example, we are printing the characters of the string.

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main (){

   char greeting[] = {'H', 'e', 'l', 'l', 'o', '\0'};

   for (int i = 0; i < 5; i++) {
      printf("%c", greeting[i]);
   }

   return 0;
}
```

Output

It will produce the following output –

```
Hello
```

Printing a String (Using %s Format Specifier)

C provides a format specifier "**%s**" which is used to print a string when you're using functions like **printf()** or **fprintf()** functions.

Example

The "%s" specifier tells the function to iterate through the array, until it encounters the null terminator (\0) and printing each character. This effectively prints the entire string represented by the character array without having to use a loop.

</> Open Compiler

```
#include <stdio.h>

int main (){

   char greeting[] = {'H', 'e', 'l', 'l', 'o', '\0'};
   printf("Greeting message: %s\n", greeting );

   return 0;
}
```

Output

It will produce the following output –

```
Greeting message: Hello
```

You can declare an oversized array and assign less number of characters, to which the C compiler has no issues. However, if the size is less than the characters in the initialization, you may get garbage values in the output.

```
char greeting[3] = {'H', 'e', 'l', 'l', 'o', '\0'};
printf("%s", greeting);
```

Constructing a String using Double Quotes

Instead of constructing a char array of individual char values in single quotation marks, and using "\0" as the last element, C lets you construct a string by enclosing the characters within double quotation marks. This method of initializing a string is more convenient, as the compiler automatically adds "\0" as the last character.

Example

</> Open Compiler

```
#include <stdio.h>

int main() {
  // Creating string
  char greeting[] = "Hello World";

  // Printing string
  printf("%s\n", greeting);

  return 0;
}
```

Output

It will produce the following output –

```
Hello World
```

String Input Using scanf()

Declaring a null-terminated string causes difficulty if you want to ask the user to input a string. You can accept one character at a time to store in each subscript of an array, with the help of a for loop –

Syntax

```
for(i = 0; i < 6; i++){
   scanf("%c", &greeting[i]);
```

```
}
greeting[i] = '\0';
```

Example

In the following example, you can input a string using scanf() function, after inputting the specific characters (5 in the following example), we are assigning null (**'\0'**) to terminate the string.

```
printf("Starting typing... ");

for (i = 0; i < 5; i++) {
  scanf("%c", &greeting[i]);
}

// Assign NULL manually
greeting[i] = '\0';

// Printing the string
printf("Value of greeting: %s\n", greeting);
```

Output

Run the code and check its output –

```
Starting typing... Hello
Value of greeting: Hello
```

Example

It is not possible to input "\0" (the null string) because it is a non-printable character. To overcome this, the "%s" format specifier is used in the scanf() statement –

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main (){
```

```
   char greeting[10];

   printf("Enter a string:\n");
   scanf("%s", greeting);

   printf("You entered: \n");
   printf("%s", greeting);

   return 0;
}
```

Output

Run the code and check its output –

```
Enter a string:
Hello
You entered:
Hello
```

Note: If the size of the array is less than the length of the input string, then it may result in situations such as garbage, data corruption, etc.

String Input with Whitespace

scanf("%s") reads characters until it encounters a whitespace (space, tab, newline, etc.) or EOF. So, if you try to input a string with multiple words (separated by whitespaces), then the C program would accept characters before the first whitespace as the input to the string.

Example

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main (){
```

```
   char greeting[20];

   printf("Enter a string:\n");
   scanf("%s", greeting);

   printf("You entered: \n");
   printf("%s", greeting);

   return 0;
}
```

Output

Run the code and check its output –

```
Enter a string:
Hello World!

You entered:
Hello
```

String Input Using gets() and fgets() Functions

To accept a string input with whitespaces in between, we should use the gets() function. It is called an unformatted console input function, defined in the "stdio.h" header file.

Example: String Input Using gets() Function

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main(){

   char name[20];

   printf("Enter a name:\n");
```

```
   gets(name);

   printf("You entered: \n");
   printf("%s", name);

   return 0;
}
```

Output

Run the code and check its output –

```
Enter a name:
Sachin Tendulkar

You entered:
Sachin Tendulkar
```

In newer versions of C, gets() has been deprecated. It is potentially a dangerous function because it doesn't perform bound checks and may result in buffer overflow.

Instead, it is advised to use the **fgets()** function.

```
fgets(char arr[], size, stream);
```

The fgets() function can be used to accept input from any input stream, such as stdin (keyboard) or FILE (file stream).

Example: String Input Using fgets() Function

The following program uses fgets() and accepts multiword input from the user.

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main(){

   char name[20];
```

```
   printf("Enter a name:\n");
   fgets(name, sizeof(name), stdin);

   printf("You entered: \n");
   printf("%s", name);

   return 0;
}
```

Output

Run the code and check its output –

```
Enter a name:
Virat Kohli

You entered:
Virat Kohli
```

Example: String Input Using scanf("%[^\n]s")

You may also use **scanf("%[^\n]s")** as an alternative. It reads the characters until a newline character ("\n") is encountered.

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main (){

   char name[20];

   printf("Enter a name: \n");
   scanf("%[^\n]s", name);

   printf("You entered \n");
   printf("%s", name);
```

```
   return 0;
}
```

Output

Run the code and check its output –

```
Enter a name:
Zaheer Khan

You entered
Zaheer Khan
```

Printing String Using puts() and fputs() Functions

We have been using printf() function with %s specifier to print a string. We can also use puts() function (deprecated in C11 and C17 versions) or fputs() function as an alternative.

Example

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main (){

   char name[20] = "Rakesh Sharma";

   printf("With puts(): \n");
   puts(name);

   printf("With fputs(): \n");
   fputs(name, stdout);

   return 0;
}
```

Output

Run the code and check its output –

```
With puts():
Harbhajan Singh

With fputs():
Harbhajan Singh
```

Array of Strings in C

In C programming language, a string is an array of character sequences terminated by NULL, it is a one-dimensional array of characters. And, the array of strings is an array of strings (character array).

What is an Array of Strings in C?

Thus, an array of strings can be defined as –

> *An array of strings is a two-dimensional array of character-type arrays where each character array (string) is null-terminated.*

To declare a string, we use the statement –

```
char string[] = {'H', 'e', 'l', 'l', 'o', '\0'};
Or
char string = "Hello";
```

Declare and Initialize an Array of Strings

To declare an array of strings, you need to declare a two-dimensional array of character types, where the first subscript is the total number of strings and the second subscript is the maximum size of each string.

To initialize an array of strings, you need to provide the multiple strings inside the double quotes separated by the commas.

Syntax

To construct an array of strings, the following syntax is used –

```
char strings [no_of_strings] [max_size_of_each_string];
```

Example

Let us declare and initialize an array of strings to store the names of 10 computer languages, each with the maximum length of 15 characters.

```
char langs [10][15] = {
   "PYTHON", "JAVASCRIPT", "PHP",
   "NODE JS", "HTML", "KOTLIN", "C++",
   "REACT JS", "RUST", "VBSCRIPT"
};
```


Printing An Array of Strings

A string can be printed using the printf() function with **%s** format specifier. To print each string of an array of strings, you can use the for loop till the number of strings.

Example

In the following example, we are declaring, initializing, and printing an array of string –

</> Open Compiler

```
#include <stdio.h>

int main (){

   char langs [10][15] = {
      "PYTHON", "JAVASCRIPT", "PHP",
      "NODE JS", "HTML", "KOTLIN", "C++",
      "REACT JS", "RUST", "VBSCRIPT"
   };

   for (int i = 0; i < 10; i++){
      printf("%s\n", langs[i]);
   }

   return 0;
}
```

Output

When you run this code, it will produce the following output –

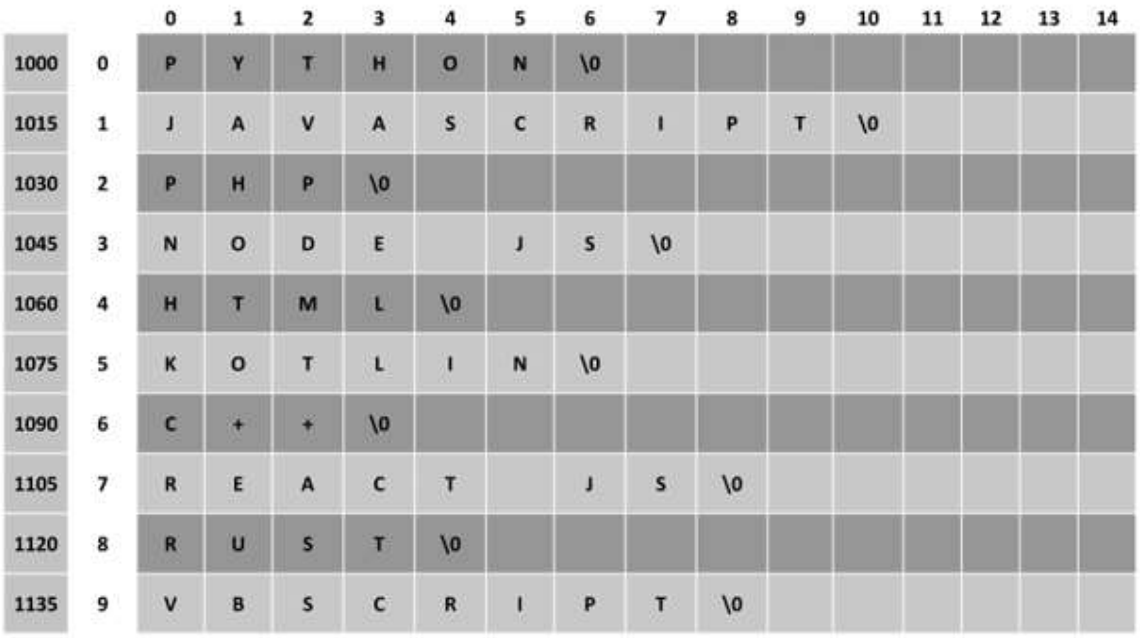
```
PYTHON
JAVASCRIPT
PHP
NODE JS
HTML
KOTLIN
C++
REACT JS
RUST
VBSCRIPT
```

Note: The size of each string is not equal to the row size in the declaration of the array. The "\0" symbol signals the termination of the string and the remaining cells in the row are empty. Thus, a substantial part of the memory allocated to the array is unused and thus wasted.

How an Array of Strings is Stored in Memory?

We know that each char type occupies 1 byte in the memory. Hence, this array will be allocated a block of 150 bytes. Although this block is contagious memory locations, each group of 15 bytes constitutes a row.

Assuming that the array is located at the memory address 1000, the logical layout of this array can be shown as in the following figure –



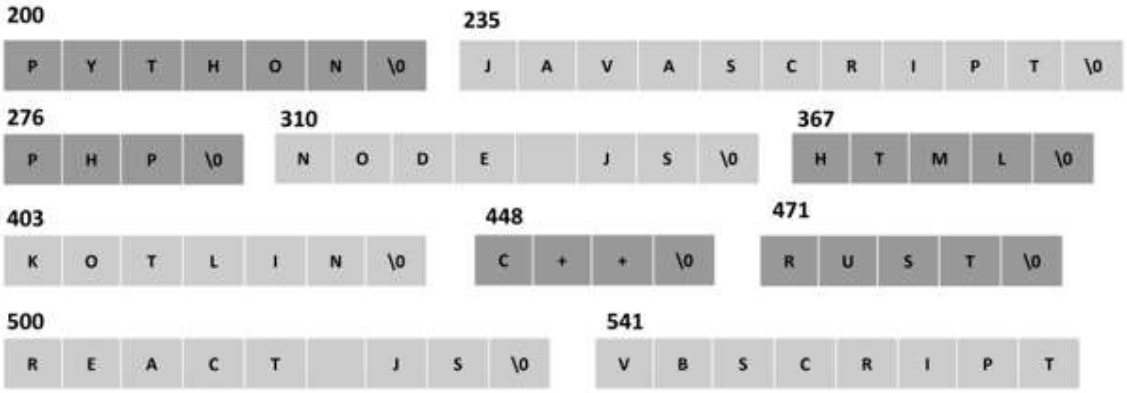
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1000	0	P	Y	T	H	O	N	\0								
1015	1	J	A	V	A	S	C	R	I	P	T	\0				
1030	2	P	H	P	\0											
1045	3	N	O	D	E		J	S	\0							
1060	4	H	T	M	L	\0										
1075	5	K	O	T	L	I	N	\0								
1090	6	C	+	+	\0											
1105	7	R	E	A	C	T		J	S	\0						
1120	8	R	U	S	T	\0										
1135	9	V	B	S	C	R	I	P	T	\0						

An Array of Strings with Pointers

To use the memory more efficiently, we can use the pointers. Instead of a 2D char array, we declare a 1D array of "char *" type.

```
char *langs[10] = {
   "PYTHON", "JAVASCRIPT", "PHP",
   "NODE JS", "HTML", "KOTLIN", "C++",
   "REACT JS", "RUST", "VBSCRIPT"
};
```

In the 2D array of characters, the strings occupied 150 bytes. As against this, in an array of pointers, the strings occupy far less number of bytes, as each string is randomly allocated memory as shown below –



Note: Here, **lang[ ]** is an array of pointers of individual strings.

0	1	2	3	4	5	6	7	8	9
200	235	276	310	367	403	448	471	500	541
5000	5004	5008	5012	5016	5020	5024	5028	5032	5036

char *langs[]

Example

We can use a **for loop** as follows to print the array of strings –

</> Open Compiler

```
#include <stdio.h>
```

```
int main(){

   char *langs[10] = {
      "PYTHON", "JAVASCRIPT", "PHP",
      "NODE JS", "HTML", "KOTLIN", "C++",
      "REACT JS", "RUST", "VBSCRIPT"
   };

   for (int i = 0; i < 10; i++)
      printf("%s\n", langs[i]);

   return 0;
}
```

Output

When you run this code, it will produce the following output –

```
PYTHON
JAVASCRIPT
PHP
NODE JS
HTML
KOTLIN
C++
REACT JS
RUST
VBSCRIPT
```

Here, **langs** is a pointer to an array of 10 strings. Therefore, if **langs[0]** points to the address 5000, then **"langs + 1"** will point to the address 5004 which stores the pointer to the second string.

Hence, we can also use the following variation of the loop to print the array of strings –

```
for(int i = 0; i < 10; i++){
   printf("%s\n", *(langs + i));
}
```

When strings are stored in array, there are a lot use cases. Let study some of the use cases.

Find the String with the Largest Length

In the following example, we store the length of first string and its position (which is "0") in the variables "l" and "p" respectively. Inside the **for** loop, we update these variables whenever a string of larger length is found.

Example

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main (){

   char langs [10][15]  = {
      "PYTHON", "JAVASCRIPT", "PHP",
      "NODE JS", "HTML", "KOTLIN", "C++",
      "REACT JS", "RUST", "VBSCRIPT"
   };

   int l = strlen(langs[0]);
   int p = 0;

   for (int i = 0; i < 10; i++){
      if (strlen(langs[i]) >= l){
         l = strlen(langs[i]);
         p = i;
      }
   }
   printf("Language with the longest name: %s Length: %d", langs[p], l);

   return 0;
}
```

Output

When you run this code, it will produce the following output –

```
Language with longest name: JAVASCRIPT Length: 10
```

Sort a String Array in Ascending Order

We need to use the strcmp() function to compare two strings. If the value of comparison of strings is greater than 0, it means the first argument string appears later than the second in alphabetical order. We then swap these two strings using the strcmp() function.

Example

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>
#include <string.h>

int main (){

   char langs [10][15] = {
      "PYTHON", "JAVASCRIPT", "PHP",
      "NODE JS", "HTML", "KOTLIN", "C++",
      "REACT JS", "RUST", "VBSCRIPT"
   };

   int i, j;
   char temp[15];
   for (i = 0; i < 9; i++){
      for (j = i + 1; j < 10; j++){
         if (strcmp(langs[i], langs[j]) > 0){
            strcpy(temp, langs[i]);
            strcpy(langs[i], langs[j]);
            strcpy(langs[j], temp);
         }
      }
   }

   for (i = 0; i < 10; i++){
      printf("%s\n", langs[i]);
   }
```

```
   return 0;
}
```

Output

When you run this code, it will produce the following output −

```
C++
HTML
JAVASCRIPT
KOTLIN
NODE JS
PHP
PYTHON
REACT JS
RUST
VBSCRIPT
```

In this chapter, we explained how you can declare an array of strings and how you can manipulate it with the help of string functions.